

Laborde Marius
Avril - Août 2026

STRI 2A

Optimisation et automatisation de l'utilisation de l'outil Dependency Track dans le cadre du maintien en condition de sécurité des systèmes satellitaires

Encadré par Aimal Mohamed et l'équipe MCS
Professeur référent : Riad Mokadem



Dependency
Track

Laborde Marius
Du 7 Avril au 7 Août 2026

Remerciements

Avant de commencer la présentation de ce rapport, je souhaite remercier l'ensemble des personnes qui m'ont aidé au succès de mes tâches et à la rédaction de ce document.

Je remercie chaleureusement Mme Laurence Laborde, cheffe de l'équipe Maintien en Condition de Sécurité MCS chez Thales Alenia Space, pour m'avoir aidé à trouver cette opportunité de stage, et m'avoir accueilli au sein de l'équipe, pour sa confiance accordée dès le début et au cœur des travaux qui m'ont été confiés.

Mes remerciements s'adressent aussi à mon tuteur de stage, Mr Aimal Mohamed, pour sa présence continue en cas de demandes, pour son aide et sa disponibilité lors de mes travaux.

Je tiens également à saluer les membres de l'équipe MCS pour leur accueil et leur soutien quotidien sur le plateau : Mr Charles Godard, Mr Nicolas Dumoulin pour le partage et l'aide donnée à propos de Dependency Track, ainsi que Mr Ryan Chambre, et Mr Aurélien Pouilles pour les échanges techniques et l'entraide sur la manipulation des inventaires logiciels (SBOMs).

Je souhaite remercier également Mr Gilles Curletto et l'équipe technique de MGI Consultant, pour leur assistance rapide et leur aide précieuse lors de la résolution de mes incidents matériels, me permettant d'avoir un environnement de travail optimal.

Enfin, j'adresse mes remerciements à mon professeur référent, Mr Riad Mokadem, ainsi qu'à l'ensemble du corps pédagogique de la formation STRI, pour la qualité de l'enseignement supérieur et le suivi de mon parcours académique et professionnel.

Sommaire

Table des matières

1 - Abstract / Résumé	5
2 - Glossaire.....	6
3 - Présentation de l'entreprise	8
3.1 - MGI Consultants	8
3.2 - Thales Alenia Space	8
3.3 - L'équipe cybersécurité et MCS	9
4 - Contexte du stage.....	10
4.1 - La vulnérabilité logicielle dans le domaine spatial	10
4.2 - Problématique	11
4.3 - Outils utilisés	11
5 - Démarche et Développement	15
5.1 - Objectifs	15
5.1.1 - Objectifs Initiaux	15
5.2 - Réalisations.....	15
5.2.1 - Configuration et prise en main du serveur Dependency Track	15
5.2.2 - Enrichissement des sources.....	18
5.2.3 - Rapports et analyses automatiques	21
5.2.4 - Mise à jour et migration vers la version 4.14.1	24
5.2.5 - Optimisation de l'utilisation du serveur Dependency track	25
5.3 - Méthodologie de travail	28
5.4 - Résultats obtenus pour MGI et Perspectives	29
5.4.1 - Résultats utilisés en production	29
5.4.2 - Avancement des autres travaux	29
5.4.3 - Mes contributions pour l'entreprise.....	30
5.4.4 - Perspectives pour la suite du stage.....	31
6 - Bilan de compétences	32
7 - Conclusion	33
8 - Bibliographie	34

9 - Table des Figures :	34
10 - Annexes	35

1 - Abstract / Résumé

Dans le cadre de mon stage de quatrième année d'études à l'école d'ingénieur Upssitech, j'ai intégré pendant quatre mois l'équipe Maintien en Condition de Sécurité (MCS) de Thales Alenia Space, au sein de MGI Consultants. L'objectif de ce stage était d'optimiser l'utilisation de l'outil Dependency-Track (DpT), plateforme logicielle libre de gestion des vulnérabilités logicielles, dans un contexte industriel spatial où la maîtrise de la sécurité des systèmes spatiaux, comprenant aussi la chaîne d'approvisionnement, est devenue cruciale.

Face à l'explosion du volume d'avis de sécurité publiés chaque année et aux limites des sources natives de l'outil Dependency-Track, j'ai conçu et développé des scripts Python couvrant l'enrichissement automatique des sources de cyber veille, l'automatisation du marquage des vulnérabilités exploitées, et l'optimisation du triage par détection de composants similaires via un algorithme de fuzzy matching. Ces travaux s'appuient sur l'API REST native de Dependency-Track et ont été intégrés dans des pipelines Gitlab CI pour une exécution automatisée. Les résultats obtenus démontrent le fort potentiel de Dependency-Track comme plateforme centrale de gestion des vulnérabilités, à condition d'être enrichie et automatisée au-delà de sa configuration initiale.

English abstract :

As part of my fourth-year engineering internship at Upssitech, I joined the Security Maintenance Service team at Thales Alenia Space, within MGI Consultants, for a 4 month period. The goal of this internship was to optimize the use of Dependency-Track, an open source software vulnerability management platform, in an industrial space context where software supply chain control has become a critical issue.

As the volume of CVEs published every year rockets and the limitations of Dependency-Track's native vulnerability sources, I designed and developed Python scripts covering the automated enrichment of cybersecurity intelligence sources. I also permitted the automated marking of exploited vulnerabilities, and the optimization of vulnerability triage through fuzzy matching algorithms for similar component detection. This work leverages Dependency-Track's native REST API and was integrated into Gitlab CI pipelines for automated execution, with a defined schedule. The results demonstrate the strong potential of Dependency-Track as a central vulnerability management platform, if it is enriched and automated beyond its initial configuration.

2 - Glossaire

ANSSI / CERT-FR : Autorité nationale française de cybersécurité et son centre d'alerte opérationnel qui publie des avis de sécurité officiels.

CISA KEV (Known Exploited Vulnerabilities) : Catalogue officiel américain recensant exclusivement les failles dont l'exploitation réelle « dans la nature » est confirmée.

CPE (Common Platform Enumeration) : Standard de nommage géré par le NIST pour l'identification syntaxique stricte des systèmes d'exploitation et des matériels (ex: cpe:2.3:a:golang:go:1.23.1)

CVE (Common Vulnerabilities and Exposures) : Identifiant unique mondial attribué à une vulnérabilité logicielle ou matérielle publique.

CVSS (Common Vulnerability Scoring System) : Standard industriel ouvert de notation (de 0 à 10) servant à évaluer la gravité théorique d'une faille.

CycloneDX : Standard de SBOM léger au format JSON ou XML, géré par l'OWASP et utilisé nativement par Dependency-Track.

Distance de Levenshtein : Algorithme mathématique mesurant la similarité entre deux chaînes de caractères, utilisé pour le regroupement fuzzy d'alias de composants.

Dry Run (Exécution à blanc) : Répétition technique d'une opération critique sur un environnement isolé pour valider les scripts et mesurer l'indisponibilité.

ENISA / CERT-EUVD : Flux européen de vulnérabilités et agence de l'Union européenne chargée de la cybersécurité.

EPSS (Exploit Prediction Scoring System) : Score probabiliste quotidien (0 à 100 %) qu'une faille soit exploitée dans la nature sous 30 jours.

MCS (Maintien en Condition de Sécurité) : Processus de veille, d'analyse d'impact, application de correctifs et de tests de non régression, pour maintenir la sécurité d'un système durant tout son cycle de vie.

NIST / NVD : Organisme gouvernemental américain et sa base de données nationale des vulnérabilités (National Vulnerability Database).

Payload (Charge utile) : Équipements embarqués à bord du satellite pour accomplir la mission technique pour laquelle il a été conçu (télécoms, observation)

PURL (Package URL) : Standard décentralisé moderne d'identification unique des paquets applicatifs ouverts, indépendant de leur écosystème (ex: pkg:npm/log4j-core@2.13.3).

RHSA (Red Hat Security Advisory) : Notifications et bulletins de sécurité émis par Red Hat décrivant les failles et les correctifs propres à ses produits.

SBOM (Software Bill of Materials) : Inventaire structuré et standardisé répertoriant l'intégralité des composants logiciels et des dépendances d'une application

TXS / TEEXMA : Logiciel et base de données relationnelle personnalisée utilisés historiquement par l'équipe MCS pour centraliser la nomenclature, les avis, les suivis de correction et les obsolescences.

VEX (Vulnerability Exploitability eXchange) : Standard complémentaire au SBOM permettant de qualifier le statut réel d'une faille dans son contexte (ex: Faux Positif, Exploitable).

Vulnerability Lookup (CIRCL/ VL) : Plateforme ouverte européenne d'agrégation et de centralisation rapide de flux de vulnérabilités.

Webhook : Mécanisme de rappel HTTP automatique permettant d'envoyer des alertes de données en temps réel vers une plateforme tierce (ex: Discord).

Exploit DB : base de données de vulnérabilités publique et open source maintenue par Offensive Security.

Score SSVC : Indique une exploitation confirmée ou un proof-of-concept public et disponible avec un état.

Fuzzy matching : technique algorithmique permettant de comparer et d'identifier des chaînes de caractères qui ne sont pas strictement identiques, mais qui présentent de fortes similitudes linguistiques ou phonétiques.

API REST : piloter à distance, via des requêtes HTTP (GET, POST, PUT), toutes les actions de Dependency-Track sans ouvrir l'interface web.

CI/CD (Continuous Integration / Continuous Deployment): méthode d'automatisation qui accompagne tout le cycle de vie du développement logiciel, via Gitlab dans le cadre du stage.

RPM : système de gestion de paquets de logiciels utilisé sur certaines distributions GNU/Linux. Le système est composé d'un format ouvert et d'un logiciel libre de manipulation des fichiers de ce format

3 - Présentation de l'entreprise

3.1 - MGI Consultants

MGI Consultants est une société spécialisée dans l'ingénierie des infrastructures informatiques, l'architecture des systèmes et la transformation vers le Cloud. Au niveau national, le groupe rassemble plus de 350 collaborateurs répartis entre son siège parisien et ses agences stratégiques de Pau, Genève et Toulouse.

L'expertise technique globale du groupe s'articule sur trois piliers technologiques majeurs :

- **Infrastructures (On-Premise et Cloud)** : Conception d'architectures hybrides, virtualisation, orchestration de conteneurs et automatisation via le scripting (DevOps).
- **Apps & Data** : Gestion, sécurisation, sauvegarde des bases de données et distribution d'applications.
- **Modern Workplace & Devices** : Gestion de la mobilité, sécurisation des terminaux et outils collaboratifs.

MGI Consultants Sud-Ouest

Mon stage de deuxième année de cursus d'ingénieur s'est déroulé spécifiquement au sein de la filiale régionale MGI Consultants Sud-Ouest.

Créée le 1er novembre 1998, cette entité sous forme de Société par Actions Simplifiée (SAS) affiche 27 ans d'ancrage et d'expertise dans son domaine d'activité : le conseil en systèmes et logiciels informatiques. Catégorisée comme Entreprise de Taille Intermédiaire (ETI), elle s'appuie sur un effectif stable de 100 à 199 salariés.

C'est depuis cette structure toulousaine que MGI Consultants déploie ses experts en cybersécurité et en gestion d'infrastructures pour accompagner les grands acteurs industriels locaux, notamment dans le secteur aérospatial à travers des missions de confiance pour le compte de Thales Alenia Space.

3.2 - Thales Alenia Space

Société commune entre Thales (67 %) et Leonardo (33 %), Thales Alenia Space (TAS) est un leader mondial de l'industrie spatiale, concevant des systèmes de haute technologie pour les télécommunications, l'observation de la Terre, la navigation et l'exploration orbitale. Avec la digitalisation du domaine spatial, la partie logicielle des liens satellites-sol et des charges utiles embarquées est devenue conséquente. Cette transformation numérique expose les architectures

à des menaces informatiques, faisant de la sécurité un enjeu de souveraineté et de résilience opérationnelle.

3.3 - L'équipe cybersécurité et MCS

Afin de garantir la robustesse à long terme des systèmes déployés, l'équipe Cybersécurité de MGI Consultants opère en équipe directe avec le service de Maintien en Condition de Sécurité (MCS) de Thales Alenia Space. Ce service s'apparente aux fonctions d'un PSIRT (Product Security Incident Response Team), dont la responsabilité est de piloter comment sont gérées les vulnérabilités affectant les produits livrés aux clients.

L'activité de cette équipe est dictée par des engagements contractuels stricts de maintien de sécurité. La mission consiste à analyser en continu la chaîne d'approvisionnement logicielle et les développements internes afin d'anticiper, neutraliser et bloquer toute tentative d'exploitation malveillante. Cette équipe opère autour d'un processus industrialisé en quatre étapes clés :

- découverte : veille active, cartographie des composants tiers via les SBOMs et identification des vulnérabilités théoriques et d'avis de sécurité, de sources différentes.
- qualification : analyse contextuelle de la menace pour distinguer les risques réels des faux positifs, en évaluant l'exposition réseau et l'exploitabilité.
- remédiation : définition des stratégies de contournement (workarounds), suppression des paquets obsolètes ou application de correctifs logiciels (patches).
- communication : Restitution et transparence vis-à-vis des parties prenantes (chefs de projets, responsables produits) et rédaction des rapports de sécurité officiels destinés aux clients finaux.

Mes missions se sont concentrées sur les deux premières étapes du processus : la découverte, à travers l'optimisation du reporting et de la centralisation des sources de veille dans Dependency-Track, et la qualification, via le développement de scripts Python, automatisant le tri, la gestion des alias et le marquage des vulnérabilités critiques.

Enfin, voici les membres de l'équipe :

- Laurence Laborde
- Aimal Mohamed
- Charles Godard
- Nicolas Dumoulin
- Ryan Chambre (alternant)
- Aurélien Pouilles (stagiaire)
- Marius Laborde (moi)

4 - Contexte du stage

4.1 - La vulnérabilité logicielle dans le domaine spatial

Une vulnérabilité est une faiblesse, une faille ou un défaut de conception présent dans un composant logiciel, un système informatique ou une infrastructure réseau. Si elle est découverte et exploitée par un attaquant, cette faille peut compromettre la sécurité du système en portant atteinte à sa confidentialité, à son intégrité ou à sa disponibilité.

Chaque année, une augmentation exponentielle de failles logicielles est présente, et la gestion de ces vulnérabilités est un défi pour les équipes de cybersécurité. Le catalogue des CVE a par exemple augmenté de plus de 180% en 10 ans, ce qui correspond à une quantité astronomique de nouvelles vulnérabilités chaque année.

Cette accélération et quantité d'alertes s'explique par la complexification des architectures logicielles, les composants logiciels libres et l'efficacité des outils utilisés pour trouver les failles. Ces outils d'audits sont parfois automatisés, mais il faut les gérer et les optimiser afin de ne pas être débordé par la quantité de nouvelles vulnérabilités. Certains outils d'intelligence artificielle pourraient être une piste, afin de limiter l'intervention humaine dans les cas de dangerosité faible.

De plus, cette accélération prend de l'importance au niveau des satellites, et dans le domaine spatial, car en effet, cela correspond aux logiciels de tous les engins en orbite, et leurs infrastructures au sol qui sont indispensables à leur fonctionnement. Les systèmes satellitaires ne sont plus isolés et sont de plus en plus digitalisés. Cette modification des systèmes génère donc plus de vulnérabilités pour les satellites, en affectant les technologies spatiales aux mêmes vulnérabilités que le reste du monde informatique.

Cette modernisation peut engendrer de nombreux problèmes comme la perte de contrôle de la charge utile, l'interception de données, ou des dénis de service, qui sont redoutés par le monde informatique, utilisant dans de nombreux cas les télécommunications via satellites.

Pour remédier aux failles découvertes, certaines contraintes sont imposées :

- fenêtre de vulnérabilité étendue : entre le moment où une CVE est publiée et le moment où un correctif peut être validé, il s'écoule un temps technique important, laissant le système exposé.
- exigence au niveau de la conception, afin d'écarter les composants et logiciels, qui sont considérés comme "à risques" ou obsolètes, en n'ayant pas de correctifs.
- prioriser les dangers : devant l'explosion du nombre de failles détectées, il est impossible de tout corriger ; l'équipe doit s'outiller pour trier les alertes et n'agir que sur les menaces dont le niveau d'exploitabilité réelle met en danger les projets.

4.2 - Problématique

Les spécificités des systèmes embarqués spatiaux imposent une rigueur critique dans le suivi des vulnérabilités et des obsolescences. Face à la multiplication des outils de veille et au risque de non-détection, nous nous posons la question suivante :

En quoi la centralisation, l'automatisation et l'enrichissement des sources au sein d'une plateforme unique comme Dependency-Track permettent-ils d'optimiser la gestion des vulnérabilités logicielles pour les environnements satellitaires ?

4.3 - Outils utilisés

Dans cette équipe, plusieurs outils fiables sont utilisés, afin de répertorier les projets et leurs composants. Voici une mise en contexte de comment sont utilisés les outils de l'équipe.

Teexma : C'est l'outil principal utilisé en production par l'équipe MCS. C'est une base de données sur mesure qui permet de centraliser et de référencer tous les produits, les composants logiciels et les avis de sécurité. Cet outil sert de support central pour générer les alertes et sortir les rapports d'analyse.

SAFE : C'est une application web utilisée en complément de la production, qui fonctionne grâce à un fichier de sortie généré par Dependency Track. Elle permet de faire l'extraction pour des avis de sécurité CERT FR et CERT IST et d'affiner l'analyse des vulnérabilités pour enrichir la base de production avant de transmettre les informations aux clients.

Dependency Track : C'est l'outil central de la détection de vulnérabilités logicielles utilisé pour importer et analyser les fichiers SBOM. Grâce à ses connexions aux bases de données comme la NVD, il permet d'identifier en continu les vulnérabilités théoriques des composants.

Sa force réside dans son API REST, qui permet d'automatiser l'importation de nouvelles sources de cyberveille et de trier les alertes plus rapidement, vu que l'intégralité des actions réalisables sur l'interface web est nativement disponible sous forme de requêtes programmables. En effet, cela lui apporte un avantage sur d'autres scanners de vulnérabilités, qui ont besoin d'agents lourds et d'accès réseaux spécifiques.

C'est cette ouverture technique qui donne la liberté de connecter automatiquement l'outil aux chaînes de production de l'entreprise (Gitlab CI), et de développer des interfaces légères pour automatiser le triage opérationnel et éliminer l'épuisement du personnel face aux alertes

Voici un exemple visuel de l'api utilisée via l'extension swagger, qui m'a permis de comprendre ce qui était nécessaire à l'utilisation de ces requêtes API :

[Figure 1 : Visualisation de certaines possibilités de l'API](#)

[Figure 2 : Visualisation d'un exemple de détails d'une requête API](#)

Dependency Track (DpT) est une plateforme logicielle libre de gestion. Issue de la chaîne de l'OWASP. DpT repose sur une analyse passive et fonctionne en 2 temps :

- Injection du SBOM : le serveur DpT reçoit un fichier au format CycloneDX qui correspond à la carte d'identité logicielle du système que l'on souhaite intégrer au projet
- DpT stocke tous ces SBOMs sous la forme de composants dans une base de données (ici PostgreSQL). Le serveur est constamment en action, utilisant un moteur de traitement (Vulnerability Analysis Task) confronte les vulnérabilités issues de sources précises (NVD...) aux composants contenus dans les projets.

Schéma du processus initial de DpT :

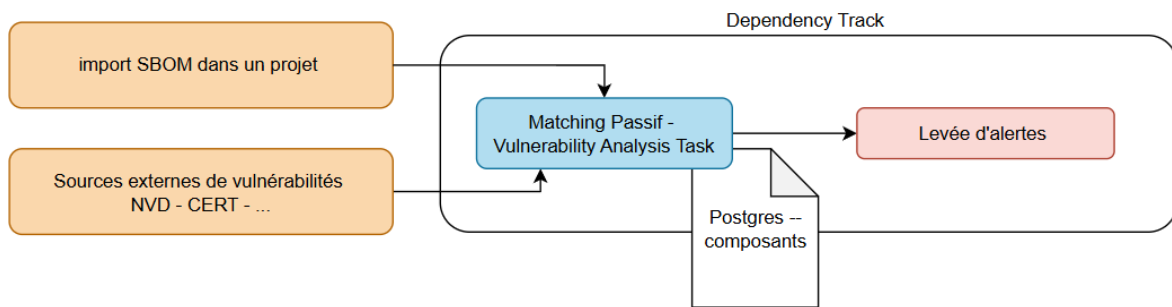


Figure 3 : Principe général de Dependency Track

La spécificité de DpT revient donc au “matching”, le croisement entre composants et vulnérabilités, et comment la “VulnerabilityAnalysisTask” identifie les composants affectés. La précision de cette identification est liée à la façon dont sont formatés les nommages des composants logiciels impactés.

- CPE (common platform enumeration) : format historique et standardisé du gouvernement américain (NVD). Il identifie une application selon une syntaxe stricte (ex: cpe:2.3:a:golang:go:1.23.1). Ce format est utile pour les OS mais s'avère trop rigide pour les bibliothèques applicatives modernes, et le code open source
- PURL (Package URL) : standard décentralisé moderne qui identifie un paquet de manière unique, indépendamment de son écosystème (ex: pkg:npm/log4j-core@2.13.3). Il intègre nativement ses métadonnées de distribution.

En production chez Thales Alenia Space, l'outil SAFE ne gère les vulnérabilités que via le format CPE. Or, la part des vulnérabilités identifiées uniquement par PURL augmente, notamment sur les paquets applicatifs modernes et libres d'utilisation. Cette limitation technique a été un facteur

déterminant dans la nécessité d'enrichir les sources de Dependency-Track au-delà de la seule source NVD.

Voici un schéma ci dessous, afin de représenter plus facilement le “matching” passif de Dependency track :

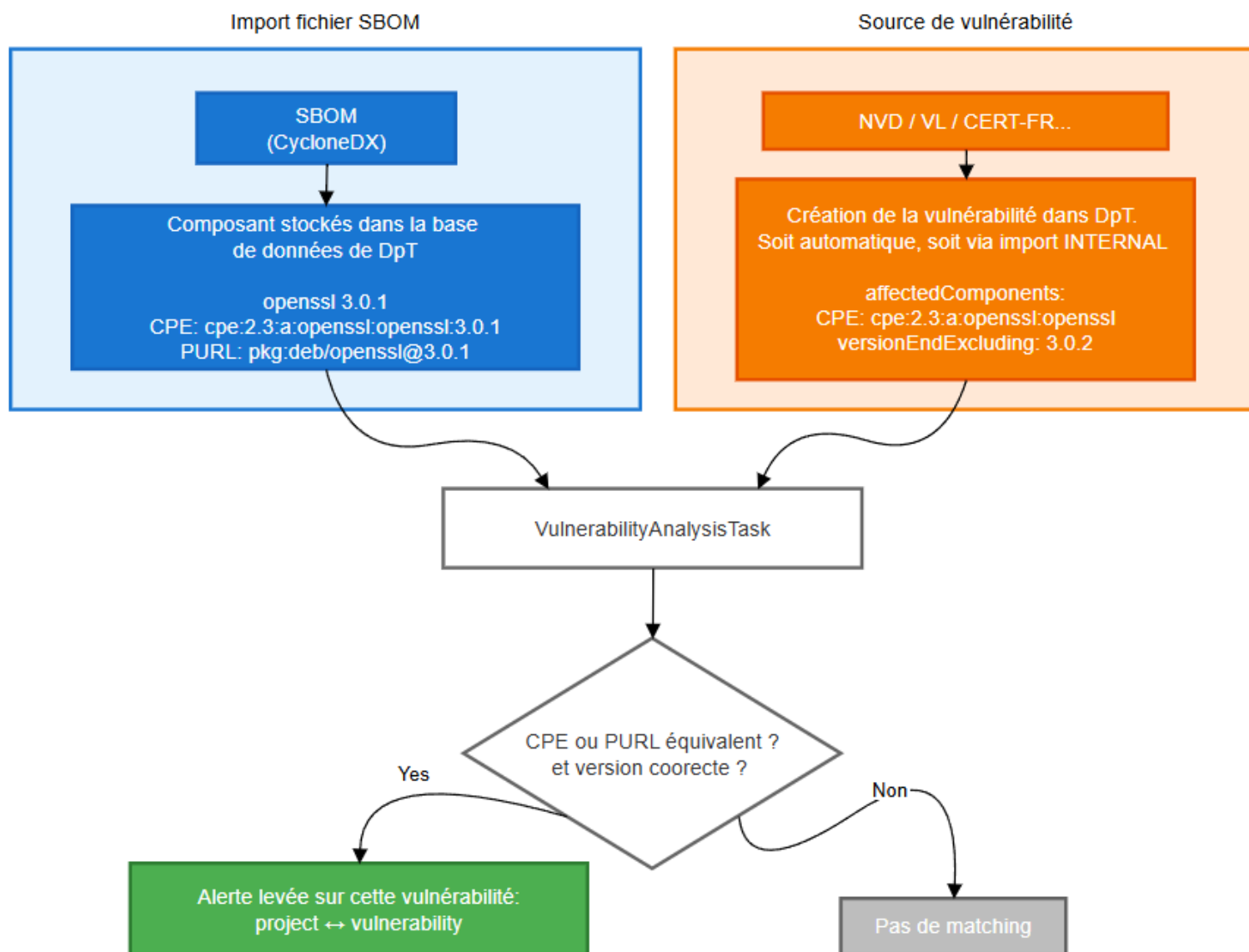


Figure 4 : Principe détaillé du matching passif de Dependency Track

Voici un tableau pour récapituler les outils présentés et illustrer les avantages et inconvénients de chacun, afin de montrer certains problèmes que les missions vont permettre de pallier :

Outil	Avantages	Inconvénients
Teexma	Base de données centrale et sur mesure pour l'équipe MCS ; référencement complet des produits et avis de sécurité ; génération des rapports clients et fiches de suivi	Outil propriétaire non automatisable via API (projet en cours pour les prochaines versions); alimentation manuelle chronophage ; pas de connexion native aux bases de vulnérabilités publiques, liés aux règles de sécurité.
Safe	Spécialisé pour les avis CERT-FR et CERT-IST ; interface de visualisation adaptée aux besoins de l'équipe. Outil interne à TAS	Correspondance CVE peu fiable et partielle ; gestion uniquement par CPE, format de moins en moins adapté plus de PURLs dans les nouvelles vulnérabilités ; ne supporte pas les fichiers SBOM ; dépend d'un fichier de sortie généré manuellement depuis DpT
Dependency track	API REST complète permettant d'automatiser via l'interface ; intégration native aux pipelines (Gitlab) ; matching passif automatique CPE et PURL sur les SBOMs importés ; open source et extensible	Source unique (NVD) ; retard fréquent de NVD sur les CVEs récentes; pas de triage assisté natif ; pas de détection des composants synonymes ou similaires

5 - Démarche et Développement

5.1 - Objectifs

Pendant mon stage, j'ai pu développer mes compétences et découvrir Dependency Track, en partant de zéro, et en développant des outils permettant d'augmenter ses atouts.

5.1.1 - Objectifs Initiaux

Voici ci-dessous un diagramme de Gantt initial, fixé et réfléchi au début du stage, proposant les principales missions à effectuer dans le temps imparti du stage. Bien évidemment, ce diagramme a vu des modifications, suite aux avancements plus ou moins rapides des missions, et aux dates courtes de l'arrivée de la soutenance du stage. La version modifiée du diagramme est dans la partie Résultats et Perspectives, et montre des objectifs nouveaux apparus pour la fin du stage, en suivant de la soutenance.

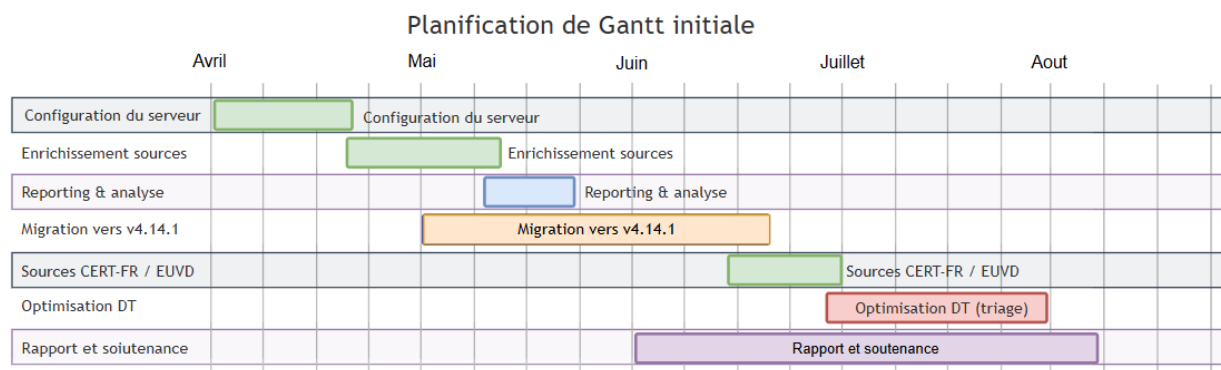


Figure 5 : Planification initiale des objectifs de stage

5.2 - Réalisations

5.2.1 - Configuration et prise en main du serveur Dependency Track

L'étape initiale du stage fut de mettre en place mon environnement de travail personnel, sur ordinateur fixe qui m'a été attribué. Cet espace de travail est consacré aux tests et développement, qui est différent du serveur de production utilisé par l'équipe. Une machine virtuelle Debian, hébergée sur l'infrastructure de MGI Consultant, via Proxmox, m'a été mise à disposition, accessible en SSH et en virtuel via l'application bureau à distance de Windows.

Afin de déployer efficacement le serveur DpT, j'ai utilisé l'installation recommandée par la [documentation officielle, via Docker](#). Cette installation fonctionne via un fichier spécifique proposé par le logiciel, où l'on peut spécifier la version de DpT à installer.

L'installation du serveur se lance en une ligne de commande et utilise trois conteneurs distincts : le serveur API, le frontend, et la base de données PostgreSQL. Le choix de la base de données est modifiable, mais non recommandé pour les dernières versions de DpT.

La configuration initiale a nécessité un ajustement à propos de la synchronisation. En effet, DpT synchronise la base NVD localement via une tâche nommée "NistMirrorTask". Ce processus est long, car beaucoup de requêtes de téléchargements sont effectuées pour le nombre conséquent de vulnérabilités (CVE) présentes. Ce blocage a été résolu en insérant une clé API NVD dans le fichier d'installation du serveur DpT, permettant au serveur d'interroger l'API NVD en temps réel plutôt que de maintenir un miroir local, réduisant ainsi le temps de démarrage opérationnel de manière significative.

Une fois l'installation du serveur opérationnelle, j'ai pu prendre en main l'interface, et comprendre comment fonctionne celui-ci. Une équipe dédiée "teammarius" a ensuite été créée dans l'interface DpT avec les droits d'administration complets, permettant de générer une clé API DpT personnelle donnant accès à l'ensemble des endpoints REST. Cette étape, puis via l'interface Swagger intégrée à DpT, a constitué le point d'entrée de la découverte de l'API, comprendre quels endpoints existent, quels payloads ils attendent, et comment les enchaîner pour automatiser des actions. Une version .txt de Swagger a été utilisée tout au long du stage comme référence technique pour valider les requêtes avant leur implémentation en Python.

Voici un schéma plus visible de mon interface de travail lié à DpT : [Figure 6 : schéma de l'organisation du serveur](#)

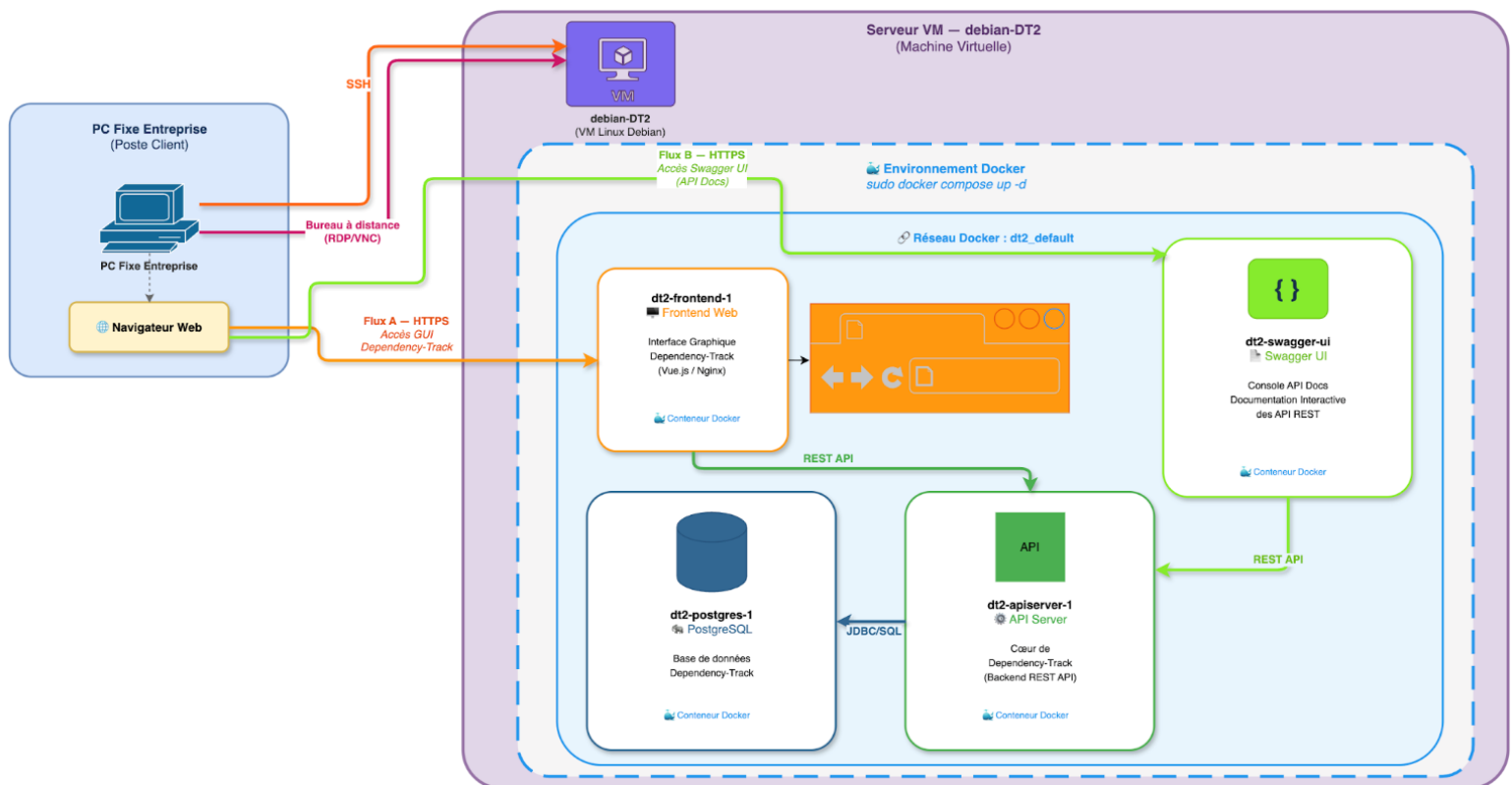


Figure 6 : Environnement de travail personnel après déploiement de Dependency Track

La prise en main de DpT s'est finalisée par la création de projets et imports de fichiers SBOMs. Cette étape a été testée via 2 méthodes :

- Génération de SBOM : En utilisant [le logiciel Syft](#) via ligne de commande, j'ai pu générer des fichiers SBOM de mon espace personnel, ce qui permet de scanner mon poste personnel par exemple.
- Import manuel : Mr Nicolas Dumoulin m'a fourni plusieurs fichiers SBOMs déjà utilisés afin de les importer.

L'import dans un nouveau projet m'a permis de visualiser le comportement natif de DpT, avec le déclenchement automatique de la "VulnerabilityAnalysisTask", l'apparition des alertes. De plus, un script mis en place par Nicolas m'a été présenté. Celui-ci effectue un import dans DpT, et sort des fichiers excels issus de l'analyse de DpT sur le fichier SBOM introduit.

Plusieurs obstacles au niveau de ce script ont été découverts. En effet, il est sensible au format, à la taille et au contenu du fichier SBOM choisi. Le serveur rejetait silencieusement les fichiers

car le format de certains identifiants et références distantes ne correspondait pas aux validations attendues. Ces anomalies bloquent l'analyse passive, empêchant la détection de vulnérabilités, sans générer d'erreurs visibles sur le serveur. Certains composants problématiques devaient être manuellement nettoyés ou corrigés afin de permettre à DpT de faire son matching.

5.2.2 - Enrichissement des sources

Initialement, la source unique de DpT est la NVD. Un des objectifs que m'avait présenté mon tuteur était d'utiliser d'autres sources externes, afin d'aller au-delà d'une unique base, et proposer une "redondance" en cas de non conformité, ou d'un retard d'une des bases. Les majeures bases connues sont :

- Github Advisories
- Google OSV
- Sonatype OSS Index
- Trivy
- VulnDB
- Snyk

5.2.2.1 Sources déjà "associées" à Dependency Track

La première étape de cette amélioration fut assez simple et rapide : l'activation de GitHub Advisories, Google OSV et Sonatype OSS Index directement depuis l'interface d'administration de DpT. Ces trois sources sont nativement supportées et ne nécessitent aucun développement. En effet, un onglet sources dans les paramètres de DpT permet de les activer rapidement, et de configurer des clés API si nécessaire. Github et Google sont deux sources qui pallient le manque de couverture des écosystèmes applicatifs modernes de la NVD.

L'intégration de Trivy a nécessité plus de travail. [Trivy](#) est un scanner de vulnérabilités open source d'Aqua Security, déployé ici en mode serveur dans un conteneur Docker sur la VM. L'objectif est de le connecter à DpT dans la partie paramètres pour qu'il analyse les composants, en complément de la base NVD. Deux problèmes principaux ont ralenti la mise en place et la fonctionnalité de Trivy :

- Isolement réseau : la configuration de réseau Docker de Trivy initiale n'est pas automatiquement liée au réseau Docker de Dependency Track. Cette configuration a dû être modifiée pour que les deux soient sur le même réseau.
- Token d'authentification : pour autoriser la communication entre les deux systèmes, un token doit être configuré des deux côtés. DpT avait un token renseigné dans ses paramètres, mais Trivy avait été lancé sans token côté serveur. La solution a été de vider le token dans la configuration DpT pour correspondre au mode sans authentification de Trivy.

Après un nouvel essai, les deux sources se sont connectées en rentrant l'adresse du réseau Docker de Trivy dans les paramètres de DpT, permettant à Trivy de remonter les vulnérabilités détectées sur les composants scannés. Dans le cadre d'un serveur de test dédié au développement, l'apport visible de Trivy reste limité, sa vraie valeur se manifesterait sur un portfolio de projets réels en production.

5.2.2.2 Sources supplémentaires à la NVD

Le deuxième enrichissement de source fut lié à Vulnerability Lookup comme source INTERNAL dans Dependency Track. Vulnerability Lookup (VL) est une plateforme européenne open source qui agrège des flux de vulnérabilités provenant de nombreuses sources et les publie sous fréquence courte tout au long de la journée.

L'idée initiale était de récupérer les dernières CVEs publiées et de les injecter dans DpT avec leurs informations CPE et PURL, pour activer le matching passif. L'ajout de ces vulnérabilités permet d'avoir les informations dans le cas où la CVE n'est pas proposée par la NVD. Dans le cas contraire, la vulnérabilité est quand même ajoutée car VL centralise des informations comme le score EPSS, le pourcentage EPSS, le CVSS Impact Subscore, les produits affectés avec des plages de version, et le lien vers la page VL de la CVE, que NVD ne fournit pas toujours.

La difficulté majeure de ces imports via Vulnerability Lookup est les formats d'entrée dans le script, récupérés par l'API de VL. En effet, initialement, le script fonctionnait en parsant un format CVE JSON v5, mais ce format varie en fonction de la sortie des nouvelles vulnérabilités, en fonction de leur provenance. Par exemple, si une vulnérabilité est issue de RedHat, alors le format utilisé sera CSAF 2.0. Ces mauvais formats ne permettaient pas au script de nourrir DpT avec les bonnes informations pour effectuer le matching passif

Face à ce problème, une modification du script fut nécessaire : un module de détection automatique du format à chaque entrée, trois parsers distincts couvrant CVE JSON v5, CSAF 2.0 et OSV ont permis de couvrir toutes les sorties. Le format OSV couvre à lui seul Bitnami, GitHub Advisory, Go Vulnerability Database Debian et d'autres sources moins connues.

De plus, un module de détection des CPE est ajouté pour le CSAF 2.0. En effet, les CPEs impactés sont décrits dans les produits portés par les nœuds parents de l'arbre et non par les sous produits portés sur les feuilles. Grâce à cela, nous pouvons alimenter le champ affected components de DpT, qui permet à DpT de faire son matching passif.

5.2.2.3 Sources Françaises et Européennes

Sous une même logique, la technique utilisée pour VL m'a permis de prendre en compte deux autres sources : les CERTFR (ANSSI) et les CERTEUVD (ENISA). Leur intégration étend l'import depuis différentes sources de vulnérabilités dans DpT, et permet de récupérer les avis de sécurité, afin de les importer dans le cas de retard de la NVD. Pour les avis CERT-FR, une découverte importante a simplifié le développement : chaque avis CERT-FR expose un endpoint

JSON officiel à l'adresse /avis/{ID}/json/, évitant entièrement le parsing HTML fragile initialement envisagé. En effet, ne possédant pas d'API spécifique pour les avis français, une source JSON d'information est bien plus pratique que du HTML, pour récupérer et importer les informations de chaque avis dans DpT. Pour CERTEUVD, le vrai endpoint de détail (/api/enisaid) a été identifié en analysant les requêtes réseau du frontend EUVD depuis les DevTools du navigateur (clic droit - inspecter - network) et il retourne les produits affectés avec leurs plages de version, des données absentes de l'endpoint liste public.

Enfin, mon tuteur travaillait sur une automatisation N8N. L'objectif est de récupérer les informations sur les avis CERT FR, via requêtes par des agents IA, afin de générer un fichier au format OSV, permettant d'importer les avis CERT FR dans DpT. C'est donc une alternative au script proposé précédemment. A sa demande, j'ai donc développé un script, issu du premier, permettant d'importer des fichiers choisis spécifiquement. Cet import est tout aussi efficace, et le résultat s'est montré positif dans Dependency track. Pour l'utilisation, ce sera donc à l'équipe de choisir le type d'import à utiliser.

Voici une représentation simplifiée de la mission d'ajout de nouvelles sources :

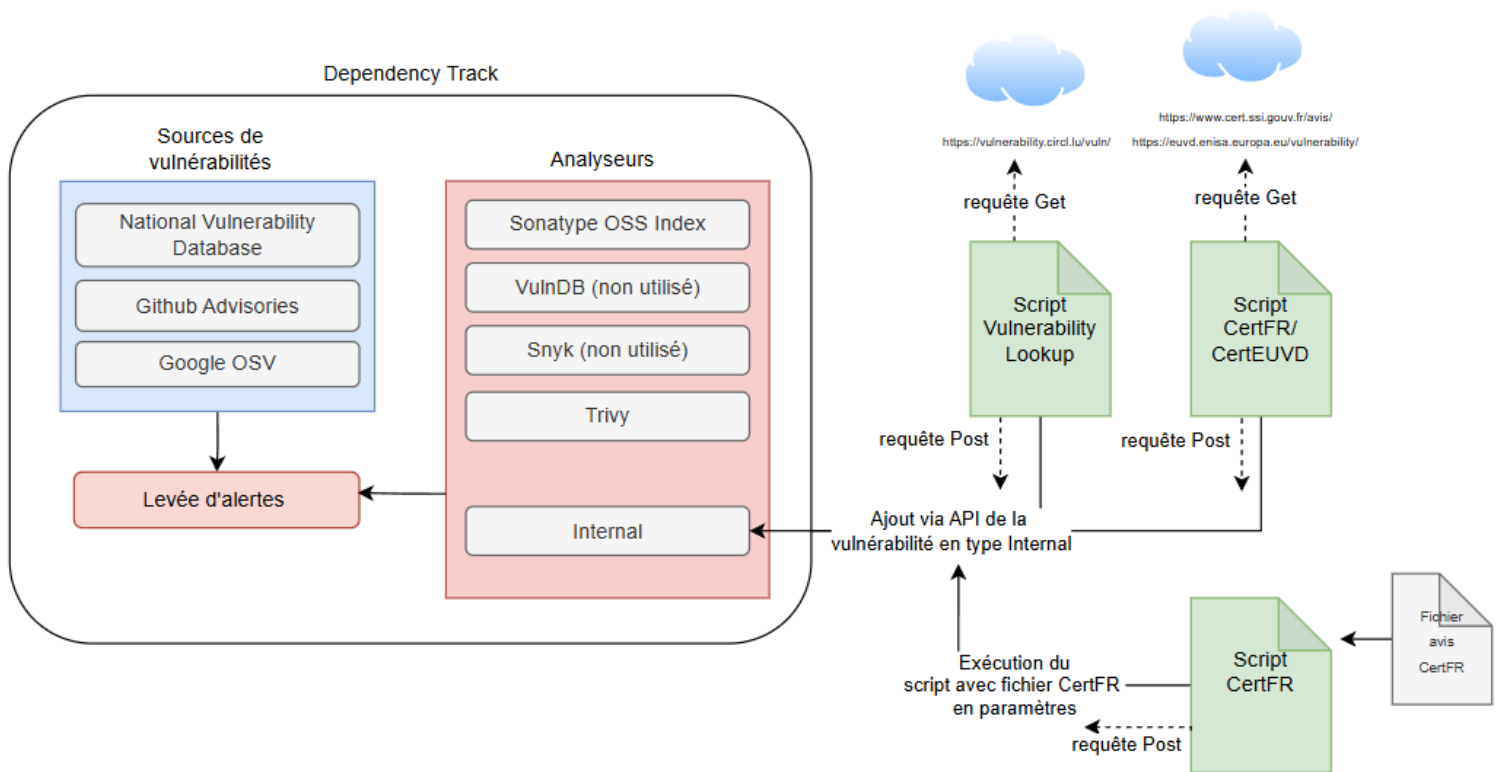


Figure 7 : Schéma simplifiant le processus d'import de vulnérabilités issues de sources différentes.

5.2.3 - Rapports et analyses automatiques

Pour l'entrée en vigueur du [Cyber Resilience Act](#), en septembre 2026, une importance catégorique doit être apportée aux produits et logiciels numériques européens, notamment dans le but de les protéger contre les attaques cyber, et de les rendre éligible aux évaluations de conformités. Le but est de responsabiliser les fabricants, les importateurs et les distributeurs et renforcer la confiance des consommateurs. Dans notre cas, cela implique qu'il est nécessaire de prendre plus en compte les vulnérabilités exploitables afin de protéger les composants et logiciels utilisés dans nos projets.

Le reporting est très important en informatique et se définit comme un processus de structuration de données techniques permettant de transformer un quantité d'informations brutes en informations claires et utilisées, comme du tri sur le nombre de sorties conséquentes de vulnérabilités de nos jours. Afin de proposer une facilitation du processus de reporting de

l'entreprise, j'ai proposé 2 scripts permettant d'automatiser 2 tâches, qui sont le marquage des vulnérabilités exploitables, et la vérification de vulnérabilités dont le score CVSS dépasse 9.0.

5.2.3.1 - Automatisation du marquage des vulnérabilités

Avec une grosse augmentation du nombre de nouvelles vulnérabilités, il est difficile de repérer celles qualifiées d'exploitables, qui sont les plus dangereuses pour les projets et systèmes supervisés. Afin de pallier ce problème et de simplifier cette vérification par l'équipe, j'ai proposé un script automatisé permettant de passer l'état d'une vulnérabilité à "Exploitable" dans Dependency Track sans intervention humaine, dès lors que son exploitation est confirmée par une source externe fiable, en 2 versions.

La première version du script utilisait le [catalogue officiel de la CISA](#), le KEV (Known Exploited Vulnerabilities), comme source de référence. Ce catalogue recense exclusivement les vulnérabilités dont l'exploitation est confirmée par les Etats-Unis. Le fonctionnement était le suivant : le script télécharge le fichier JSON du catalogue CISA, compare chaque CVE listée avec les vulnérabilités présentes dans les projets DpT, et marque automatiquement, via pipeline Gitlab, en "Exploitable" les findings correspondants via l'API PUT /v1/analysis, en y ajoutant un commentaire qui spécifie l'action avec l'heure détaillée. Une comparaison assez simple évite de re-télécharger le fichier si aucune mise à jour n'a eu lieu depuis le dernier lancement, en vérifiant la date et l'heure du fichier JSON récupéré.

Cette première version présentait cependant une limitation importante : le catalogue de [cisa.org](#) n'est pas mis à jour régulièrement, et possède une petite quantité de vulnérabilités (~1500), ce qui crée un laps de temps assez problématique entre le moment où une vulnérabilité commence à être exploitée et le moment où elle est annoncée dans le fichier JSON.

La décision a donc été prise de migrer vers Vulnerability Lookup comme source, mais à jour dans des délais bien plus courts.

La difficulté de cette migration était de ne pas prendre toutes les nouvelles vulnérabilités... : VL les publie toutes, pas uniquement les exploitées. Le script se déclenche uniquement lorsque les données confirment explicitement l'exploitation active, via deux mécanismes distincts.

Le premier est un bloc ADP de type KEV, intégré dans les données VL lorsque la CISA a référencé la vulnérabilité dans son catalogue (version 1 du script), ce qui préserve la fiabilité de la qualification.

Le second est un indicateur SSVC avec le statut "active" ou "poc", fourni par des organismes tels que la CISA-ADP. Le fait d'utiliser ces sources nous permet d'avoir plus de fiabilité sur la sortie des vulnérabilités, et sur leur date de sortie, qui ont moins de chances d'être retardées.

5.2.3.2 - Script de veille sur les vulnérabilités critiques

En parallèle du marquage automatique, un second besoin a été identifié : être alerté rapidement lorsqu'une nouvelle vulnérabilité critique est publiée, même si elle ne concerne pas directement les projets présents dans DpT. Le script de veille est là pour ça, en récupérant les dernières vulnérabilités depuis le flux Vulnerability Lookup et en filtrant celles dont le score CVSS dépasse 9.0, par demande des membres de l'équipe.

Pour chaque vulnérabilité retenue, le script extrait les informations essentielles à une qualification rapide :

- identifiant
- CVE,
- score CVSS,
- vendeurs et produits affectés, dates de publication et de modification,
- description,
- lien direct vers la page Vulnerability Lookup.

Deux modifications ont été ajoutées pour augmenter la valeur du rapport.

Le premier consiste à détecter si la vulnérabilité est marquée comme activement exploitée dans les données VL, en utilisant la même logique KEV que le script présenté précédemment.

Le second enrichissement interroge l'API publique d'Exploit-DB pour chaque CVE retenue, afin de vérifier si un exploit public est disponible et téléchargeable, ce qui représente une information directement actionnable pour l'équipe MCS. Si des exploits sont trouvés, des infos et un lien direct vers la page Exploit-DB sont ajoutés au rapport.

Pour les deux scripts, les résultats sont compilés dans un fichier texte daté, avec une vérification anti-doublon évitant de signaler plusieurs fois la même CVE entre deux lancements.

Le rapport est ensuite envoyé automatiquement sur un canal Discord via Webhook, permettant à l'équipe d'être notifiée en temps réel sans avoir à surveiller manuellement les sorties du script. Ce script a été intégré dans un pipeline Gitlab CI avec une exécution planifiée, garantissant une veille continue et automatisée.

Voici ci-dessous une représentation de mes propositions de « reporting », qui se sont avérées concluantes.

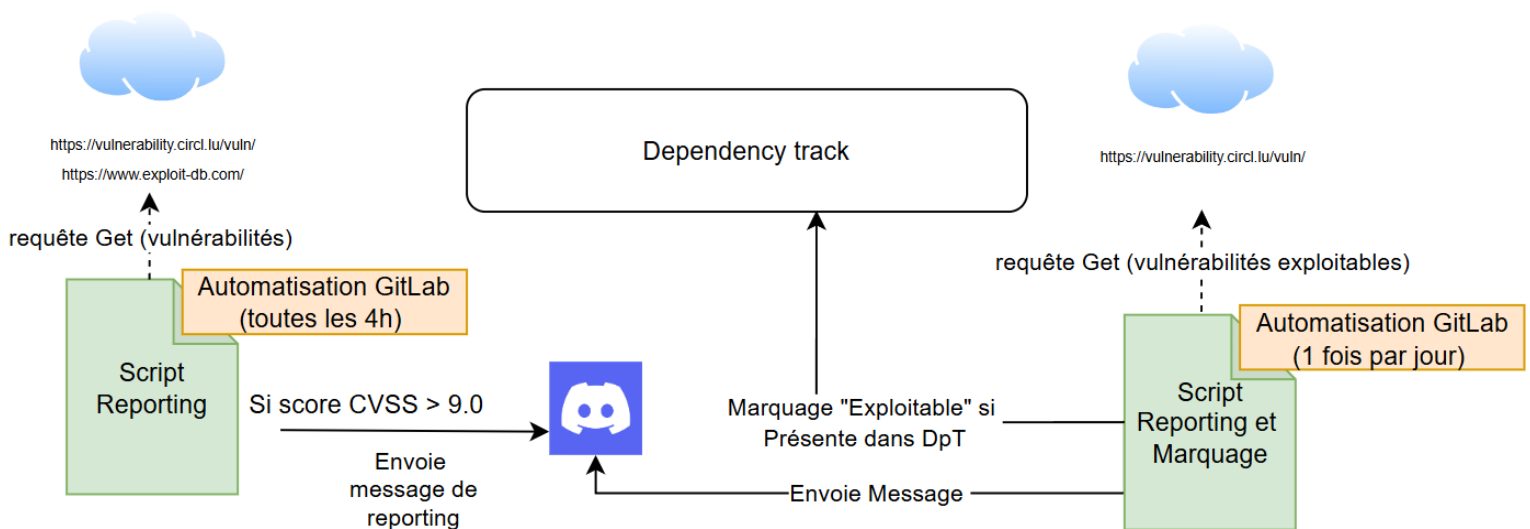


Figure 8 : Schéma d'explication des scripts de veille/marquage

5.2.4 - Mise à jour et migration vers la [version 4.14.1](#)

En parallèle des développements, un objectif pré-fixé concerne la mise à jour du serveur Dependency Track, qui tournait alors en version 4.12.0. La v4.14.1 apportait des correctifs importants, notamment le hash des clés API en base de données et la suppression de l'accès non authentifié aux badges de statut. Une mise à jour directe en production n'est pas un bon exemple à suivre. J'ai donc proposé et mis en place une [procédure de Dry Run](#) (mise à jour après copie) structurée en trois phases avant la phase sur le serveur réel.

5.2.4.1 - [Phase 1 : Serveur de test via copie, avec données de test](#)

La première phase consistait à valider la procédure sur mon serveur Dependency track. Deux volumes Docker ont été créés pour recevoir une copie des données du serveur de test existant, soit un pour la base PostgreSQL et l'autre pour les fichiers applicatifs de DpT (clés de chiffrement, index). Cette copie a été réalisée via un conteneur temporaire Alpine, sans bloquer le travail actuel du serveur. Un fichier docker-compose dédié au Dry Run a ensuite été configuré pour pointer vers ces volumes copiés et déployer les images DpT en version 4.14.1. Le démarrage s'est déroulé sans erreur, les trois conteneurs ont atteint l'état Healthy, et l'interface affichait bien la version 4.14.1 avec toutes les données du serveur de test intactes.

5.2.4.2 - Phase 2 : Qualification sur données de production réelles

Cette phase est la plus critique. Le serveur de production (192.168.90.13) hébergeait environ 21 Go de base de données réelle. Les fichiers applicatifs ont été compressés sur le serveur de production puis transférés via SCP vers le serveur de test. Pour la base de données, afin d'éviter de saturer l'espace disque de la production, le transfert a été effectué en streaming direct via un tunnel SSH, sans création d'un fichier intermédiaire, qui aurait saturé les capacités de la machine de test. Malgré une alerte temporaire "Unhealthy" due au temps de traitement des 21 Go, les logs ont confirmé le succès de la migration et l'interface affichait les mêmes données que la production, en v4.14.1.

5.2.4.3 - Phase 3 : Migration du serveur de production

Vu que les 2 précédentes phases furent positives, la migration du serveur de production a pu être réalisée sans problèmes. Les conteneurs ont été arrêtés, les sauvegardes envoyées en tunnel SSH vers le serveur de test pour ne pas saturer l'espace disque de la production (limité à 92 Go), puis le fichier docker-compose.yml a été modifié pour pointer vers les images v4.14.1. Le lancement de la commande docker-compose up -d a déclenché automatiquement la migration du schéma de base de données. Un plan de rollback avait été préparé en cas d'échec, permettant de restaurer les anciennes images et la base de données sauvegardée en quelques minutes. Cette procédure m'a aussi permis de découvrir des changements de comportement entre les versions, notamment les nouvelles permissions requises pour les badges et la gestion des tags, que j'ai documentées et transmises à l'équipe pour mise à jour de leur configuration.

Les améliorations principales de la version v4.14.1 sont les suivantes. Le moteur de comparaison de versions a été amélioré, ce qui permet de limiter le nombre de faux positifs, sur les composants avec des versions dont le format n'est pas forcément bien écrit. Le tri des paquets RPM a été amélioré avec une meilleure analyse des PURL spécifiques aux distributions Linux (distro et arch). Cette v4.14 est un atout pour TAS qui nécessite de la précision sur leurs PURL et sur leur quantité de paquets RPM.

5.2.5 - Optimisation de l'utilisation du serveur Dependency track

Au fil du stage, et après explication par mon tuteur, les limites de DpT se sont rendues visibles, notamment par rapport aux noms des composants. En effet la majeure partie des composants, issus de SBOMs peuvent ne pas avoir les mêmes noms. DpT n'est pas en mesure de gérer son inventaire en repérant les synonymes, ou les petites variantes de noms. Un paquet peut être présent sous 3 formes différentes et être non détecté par DpT. Par exemple pyyaml, python3-pyyaml et PyYAML, est traité comme trois composants distincts, générant pour DpT 3 choses différentes, donc 3 détections par la tâche de matching passif. De plus, si l'objectif est d'utiliser l'interface de DpT, la décision de l'utilisateur sur un composant aurait été dupliquée, pour gérer la décision pour toutes les variantes.

5.2.5.1 - Analyser les similarités des composants

Afin d'analyser les composants qui se ressemblent et les liés entre eux, je me suis penché sur deux points, présentés par mon tuteur, qui sont le fuzzy matching et la gestion des aliases.

- Fuzzy matching : algorithme d'approximation linguistique (comme la distance de Levenshtein) permettant de détecter et de regrouper les composants logiciels malgré des variations
- Gestion aliases : La gestion d'alias est une technique d'ingénierie qui consiste à associer différentes variantes de nommage d'un même composant logiciel sous une identité unique de référence.

Afin de tester ces technologies, j'ai fait un script python permettant de les visualiser. Tout d'abord, la détection d'aliases certifiés via le dictionnaire CPE de la NVD (deux composants partageant le même couple vendor:product dans le dictionnaire sont des synonymes confirmés). Pour le fuzzy matching, le script détecte les similarités de noms par distance de Levenshtein sur les noms normalisés. Un seuil de similarité configurable permet d'ajuster la sensibilité, placé sur 90% de similarité par défaut (choix personnel).

Une fois le script effectué, un rapport est généré sortant un récapitulatif du scan effectué sur un projet choisi. Au-delà de la "préparation au triage", ce script peut être utilisé de façon autonome pour avoir une vue d'ensemble de la qualité et de la cohérence d'un inventaire de projet.

5.2.5.2 - Outil de triage interactif, version 1

Afin de simplifier l'utilisation de DpT, une idée m'est venue et sous proposition à l'équipe, j'ai pu développer un script de triage des vulnérabilités détectées sur Dependency track : qualifier rapidement un volume important de vulnérabilités sans passer par l'interface web de DpT et trop de clics.

Le script vex_triage.py v1 propose une session de triage interactive en ligne de commande, projet par projet. Les findings sont récupérés via l'API DpT et triés automatiquement par sévérité décroissante (CRITICAL = 9.0 ou plus en premier). Pour chaque finding, un dossier enrichi est affiché :

- identifiant
- CVE,
- composant
- version concernés,
- score CVSS, score EPSS avec pourcentage récupéré en temps réel depuis l'API FIRST, source de détection,
- liens directs vers NVD et Vulnerability Lookup,
- état actuel du finding dans DpT.

L'utilisateur peut donc choisir le nouvel état de la vulnérabilité en fonction de son choix :

- **f** pour Faux Positif,
- **n** pour Non Concerné,
- **e** pour Exploitable,
- **t** pour En Triage,
- **s** pour passer,
- **q** pour quitter

La décision est écrite immédiatement dans DpT via requête api PUT /v1/analysis avec un commentaire horodaté. Un fichier de rapport texte trace chaque action en temps réel, permettant de reprendre une session interrompue sans perdre la progression. Le script a été testé sur le serveur de test avec des projets réels et fonctionne correctement.

5.2.5.3 - Triage version 2 avec intégration du fuzzy matching

La version 2 fusionne les deux scripts précédents en un outil unique. Avant de commencer le triage, le script exécute automatiquement la phase d'analyse fuzzy sur tous les composants du projet, enrichissement NVD si la clé API est configurée, puis groupage CPE et Levenshtein. Un résumé des groupes détectés est affiché avant le premier finding.

Pendant le triage, si le composant concerné par un finding appartient à un groupe, un avertissement discret en jaune indique les composants similaires identifiés. Après chaque décision, si un groupe existe, le script propose de propager la décision à tous les membres du groupe ayant le même finding CVE, l'analyste confirme ou refuse en une touche, et la propagation est tracée dans le rapport.

Cette approche réduit concrètement le nombre de décisions manuelles nécessaires sur les grands inventaires : une décision sur pyyaml peut se propager instantanément à python3-pyyaml et PyYAML sans intervention supplémentaire. Le rapport de session contient deux sections, le résumé de la phase fuzzy et le journal de triage avec les propagations explicitement notées.

Voici en lien deux images permettant de visualiser l'interface proposée pour le triage, qui remplace tous les boutons à utiliser dans l'interface du projet et d'audit d'une vulnérabilité.

[Figure 9 : interface du script de tri](#)

5.3 - Méthodologie de travail

L'intégralité de mon stage s'est déroulée sur deux principes d'organisations différents, qui sont la traçabilité technique et la réaction/agilité face aux besoins de l'équipe et aux nouvelles sorties d'informations.

Afin de garder une trace propre de l'étendu des réalisations, et éventuellement fournir des documents si nécessaires, j'ai formalisé une documentation continue. Ce choix me permet de structurer l'avancement de mes tâches, et d'être à jour sur les documents, pour que mon travail durant le stage profite à l'équipe, dans le cas de réutilisation de mes travaux. Par exemple, des documents expliquant chaque script, des détails sur les fonctions des scripts et les procédures de mise à jour du serveur.

Des réunions hebdomadaires en équipe MCS, les lundis, ont rythmé le stage d'une manière différente des objectifs initiaux, car de nouveaux points d'alignements ont permis d'adapter les priorités en fonction de l'actualité des menaces cyber, comme l'intégration de Vulnerability Lookup comme source pour DpT.

La gestion et la documentation du travail ont été décentralisées, en combinant Google Drive et l'utilisation de Gitlab pour les pipelines d'automatisation de certains scripts. Cette approche m'a permis de maintenir un accès distant à mes travaux, notamment lors des déplacements sur site chez TAS. Cette organisation a démontré son efficacité partielle lors d'une défaillance matérielle survenue en cours de stage : grâce à la synchronisation Drive, plus de 90% des travaux ont pu être récupérés immédiatement sans interruption significative de l'activité. Cet incident a mis en évidence une chose : une intégration complète à Gitlab, au-delà des seuls pipelines CI/CD, permettrait de centraliser l'ensemble du code, de la documentation et des fichiers de configuration dans un environnement versionné, traçable et résilient, pleinement adapté aux pratiques de développement et d'automatisation mises en œuvre durant ce stage.

5.4 - Résultats obtenus pour MGI et Perspectives

Les missions accomplies ont apporté de nombreux résultats positifs pour l'équipe et des perspectives d'améliorations possibles. Voici tout d'abord un diagramme de Gantt, récapitulatif des missions effectuées durant la période du stage, différent de l'initial, dû à mon avancement :

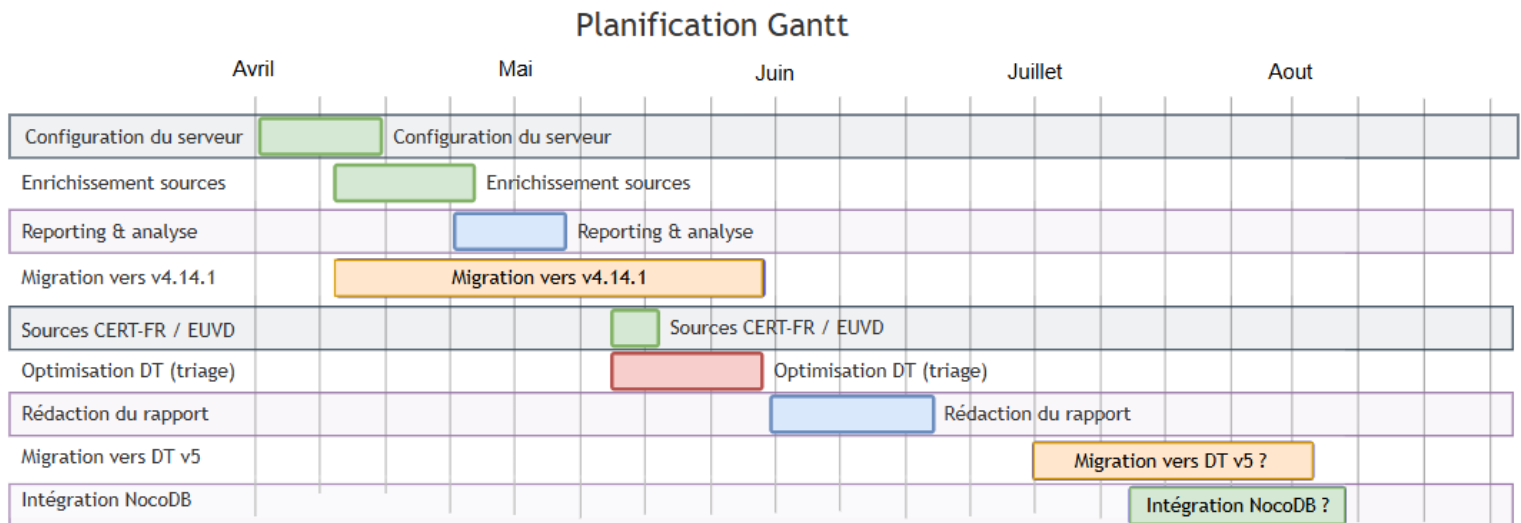


Figure 10 : Gantt des missions

5.4.1 - Résultats utilisés en production

A ce jour, deux réalisations de scripts ont été acceptées et sont utilisées actuellement en production par l'équipe, comprenant le script de veille/notification pour les vulnérabilités avec un score cvss dépassant le 9.0, et la version v4.14 de DpT, après la migration de la version v4.12. Ces deux éléments montrent une valeur ajoutée pour l'équipe, au-delà du serveur de tests.

5.4.2 - Avancement des autres travaux

Pour ce qui est des autres scripts, ils sont fonctionnels et testés sur le serveur de test, mais n'ont pas encore été déployés sur le serveur de production. Leur passage en production dépend désormais d'une présentation détaillée à l'équipe, déjà entamée lors des réunions hebdomadaires sous forme d'aperçu rapide, mais qui nécessite une session dédiée pour expliquer concrètement leur prise en main, transmettre le code source et la documentation associée, et répondre aux questions opérationnelles de l'équipe avant adoption.

Pour citer un exemple, j'ai demandé une modification de script de production, afin que cela soit adapté à mon travail. En effet le script de triage interactif modifie l'état des vulnérabilités sur

Dependency Track, mais celui-ci n'est pas visible dans les fichiers d'extractions générés par le script de Mr Nicolas Dumoulin. Après échange avec lui, il a pu ajouter la colonne "état" dans le fichier Excel de sortie. Cette amélioration ajoute des informations supplémentaires utiles pour les autres outils, qui utilisent les sorties Excel de DpT.

5.4.3 - Mes contributions pour l'entreprise

En prenant du recul sur l'ensemble du stage, l'apport principal à l'équipe MCS se situe sur trois axes complémentaires.

Le premier est le gain de temps opérationnel : l'automatisation du marquage des vulnérabilités exploitables et du triage assisté réduit considérablement le temps consacré aux tâches répétitives de qualification, qui occupaient auparavant une part importante du travail manuel sur l'interface de Dependency-Track.

Le second axe est la réduction du risque : la migration de la source de référence pour les vulnérabilités exploitées, du catalogue CISA vers Vulnerability Lookup, réduit la fenêtre d'exposition entre la découverte d'une exploitation active et sa prise en compte dans les projets de l'équipe, ce qui répond directement à la problématique de fenêtre de vulnérabilité étendue identifiée en introduction de ce rapport.

Le troisième axe est la centralisation des actions : l'enrichissement de DpT avec de nouvelles sources et de nouveaux outils de triage réduit la dépendance de l'équipe à des outils externes peu automatisables comme SAFE, en consolidant progressivement le travail de qualification autour d'une plateforme unique pilotable par API.

Voici en lien des images et une petite description sur chaque permettant de visualiser l'interface de Dependency track et les quelques résultats obtenus visibles :

[Figure 11 : Interface Dependency track et visualisation de résultats](#)

De plus, la majorité des inconvénients identifiés pour Dependency-Track, à savoir la dépendance à une source unique, le retard de la NVD et l'absence d'aide au tri pour un grand nombre de vulnérabilités, ont été traités au cours du stage grâce à des outils dont certains sont déjà en production et d'autres en attente de déploiement. Si l'on devait résumer la stratégie, elle serait de recentrer progressivement les actions sur Dependency Track plutôt qu'à corriger les outils directement, chacun de leur côté.

5.4.4 - Perspectives pour la suite du stage

Pour la suite du stage, sur 1 mois restant, quelques objectifs et perspectives sortent du lot. Il faut transmettre et présenter mes résultats à l'équipe, pour éventuellement en voir l'utilisation dans DpT de production. Ensuite, un axe d'amélioration conséquent a été vu pour la version v5 de Dependency Track qui a été annoncée récemment. Cette mise à jour aurait de nombreux atouts, car les nouveautés permettent à DpT d'être plus fiable et optimisé :

- PostgreSQL 14+ obligatoire pour fluidifier le traitement des gros volumes de données, et couper les bases obsolètes.
- Conteneurs Docker uniquement, ce qui supprime définitivement les anciens fichiers de déploiement .war.
- Migration manuelle stricte : Le passage vers la v5 a besoin de l'utilisation du nouvel outil v4-migrator, imposant que l'instance de production actuelle soit mise hors ligne et qu'elle ait été préalablement mise à jour au minimum en version 4.14.2.

Ce dernier point est important pour l'avenir du serveur DpT et une bonne prise en main de la procédure de migration est importante.

Un second axe concerne l'automatisation N8N développée par mon tuteur, qui génère actuellement des fichiers au format OSV à partir des avis CERT-FR. Le script d'import développé durant le stage pourrait être directement intégré à ce flux N8N, de façon à ce que la génération du fichier et son import dans DpT s'effectuent en une seule étape automatisée, sans intervention manuelle entre les deux. Enfin, l'outil NocoDB, pourrait être exploré comme piste de centralisation entre les différents scripts développés au cours du stage.

6 - Bilan de compétences

L'ensemble du stage m'a beaucoup apporté et j'ai apprécié l'ensemble de l'équipe MCS, en voici un tableau montrant les compétences développées selon 3 aspects

Savoir	- Fonctionnement interne de DpT - Standards de l'écosystème des vulnérabilités (CVE, CVSS, EPSS, SBOM) et exploits - Formats des données utilisées (cve json 5, csaf 2.0, osv) - Architecture et déploiement sur Docker - Enjeux de cybersécurité dans le spatial, et découvertes des lois en matière de sécurité Cyber Resilience Act
Savoir-faire	- Développement python orienté avec API REST (authentification, parsing...) - Administration et migration d'un serveur de production sur machines virtuelles - Intégration de pipelines programmées sur Gitlab CI - Documentation et recherches sur les API et leur endpoints des sources CERTFR, CERTEUVD - Documentation technique sur toutes les missions, garder une trace claire.
Savoir-être	- Autonomie de gestion du planning - Priorisation des tâches sans feuille de route imposée, en fonction des nouvelles sorties d'informations de l'équipe. - Communication avec beaucoup de membres d'équipes sur l'avancement du projet et la présentation de mes travaux. - Adaptabilité face aux incidents matériels - Sollicitation d'aide dans le besoin

Les catégories développées ne sont pas considérées indépendantes car leur addition permet de passer une bonne expérience lors d'un stage, la plupart du savoir-faire a été rendu possible grâce au savoir théorique appris. De plus, l'écoute de mon tuteur et de l'équipe a facilité le savoir-être.

7 - Conclusion

L'objectif initial de ce stage était d'optimiser un serveur Dependency track avec des limitations, au sein de l'équipe MCS de TAS. Dans un contexte où une forte augmentation de nouvelles vulnérabilités est présente, la problématique a trouvé une réponse concrète à travers le développement de scripts python permettant d'utiliser à son potentiel Dependency track, grâce à son API. Ces améliorations permettent de couvrir le cycle de vie d'une vulnérabilité et d'aider considérablement l'utilisation de Dependency track.

Parmi ces réalisations, deux sont déjà utilisées en production, ce qui constitue un bon avancement de la démarche engagée. Les autres scripts/outils, sont fonctionnels et prêts à la présentation auprès de l'équipe, et ouvrent une perspective sur l'utilisation plus poussée du serveur de production.

Au-delà de l'aspect technique, le stage confirme qu'un outil comme Dependency Track possède une valeur qui peut être améliorée, si les capacités le permettent et qu'une automatisation est possible pour en faciliter son utilisation. En fonction du contexte, il faut juger à quelle proportion on peut automatiser un outil, sans remettre en question la sécurité des projets, et en gardant une supervision humaine sur les actions. La sortie annoncée de la version 5 de DpT, avec ses exigences de migration renforcées, confirme par ailleurs que cette démarche d'optimisation continue devra se poursuivre dans la durée, bien après la fin de ce stage.

Merci d'avoir pris le temps de lire mon rapport de stage.

8 - Bibliographie

CERT-EUVD : <https://euvidservices.enisa.europa.eu/> (Consulté le 21 mai 2026)
CERT-FR : <https://www.cert.ssi.gouv.fr/> (Consulté le 13 mai 2026)
CERT-IST : <https://www.cert-ist.com> (Consulté le 13 mai 2026)
CISA KEV : <https://www.cisa.gov/known-exploited-vulnerabilities-catalog> (Consulté le 21 avril 2026)
CIRCL / Vulnerability Lookup : <https://vulnerability.circl.lu/> (Consulté le 28 avril 2026)
CVSS (FIRST) : <https://www.first.org/cvss/> (Consulté le 16 avril 2026)
CycloneDX : <https://cyclonedx.org/> (Consulté le 8 avril 2026)
Docker : <https://www.docker.com/> (Consulté le 8 avril 2026)
Dependency-Track : <https://docs.dependencytrack.org/> (Consulté le 7 avril 2026)
EPSS (FIRST) : <https://www.first.org/epss/> (Consulté le 24 juin 2026)
GitHub Advisories : <https://github.com/advisories> (Consulté le 20 avril 2026)
Google OSV : <https://osv.dev/> (Consulté le 20 avril 2026)
NVD / NIST : <https://nvd.nist.gov/> (Consulté le 9 avril 2026)
TEEXMA (Bassetti) : <https://www.bassetti-group.com/fr/teexma/> (Consulté le 23 avril 2026)
Trivy Server : <https://aquasecurity.github.io/trivy/latest/> (Consulté le 20 avril 2026)
Exploit-DB : <https://www.exploit-db.com/> (Consulté le 20 mai 2026)
First SSVC (score et déclaration "poc"/"active") : <https://www.first.org/> (Consulté le 24 juin 2026)
NocoDB : <https://nocodb.com/> (Consulté le 28 avril 2026)

9 - Table des Figures :

- **Figure 1** : Cartographie générale des fonctionnalités exposées par l'API REST de Dependency-Track (Lien d'accès technique)
- **Figure 2** : Syntaxe et structure de données d'une charge utile (payload) pour requête d'analyse (Lien d'accès technique)
- **Figure 3** : Architecture générale du flux d'ingestion des SBOMs et du traitement passif du serveur (Schéma intégré)
- **Figure 4** : Logique décisionnelle détaillée de la Vulnerability Analysis Task (Corrélation CPE et PURL) (Schéma intégré)
- **Figure 5** : Planification de Gantt initiale pour le stage
- **Figure 6** : Topologie réseau de l'environnement de développement personnel sous Debian et Proxmox (Schéma intégré)
- **Figure 7** : Flux d'interconnexion et d'ingestion automatique des sources de cyber veille souveraines (CTI) (Schéma intégré)
- **Figure 8** : Schéma architectural de l'automatisation des pipelines Gitlab CI et du système d'alerte Webhook (Schéma intégré)
- **Figure 9** : Interface en ligne de commande (CLI) et console de triage interactif VEX (Lien d'accès technique)

- **Figure 10** : Diagramme de Gantt des réalisations techniques et des perspectives d'ouvertures du stage (Schéma intégré)
- **Figure 11** : Interface d'administration globale de Dependency-Track et restitution des indicateurs de risques (Lien d'accès aux captures)

10 - Annexes

[Annexe 1 : Document lié au Dry Run pour la migration vers la version 4.14.1 de DpT](#)

[Annexe 2 : Document de déroulement de la migration](#)

[Annexe 3 : Document de propositions des scripts fait](#)

[Annexe 4 : Document d'explication de la mise en place de Gitlab pour automatisation des marquages KEV](#)

[Annexe 5 : Document d'explication de la mise en place du script de veille, détection CVE avec score > 9.0](#)

[Annexe 6 : Codes python des scripts - Lien vers drive google contenant tous les codes pythons](#)

[Annexe 7 : readme d'utilisation des scripts - documentation du travail réalisé](#)

[Annexe 8 : images proposées dans le rapport pour plus de visibilité](#)